

CRESCENDO ENGINEERING DOCUMENTATION

Project Architecture & Functionality Audit

A simplified but detailed guide to understand, present, and explain the full React web application quickly.

PROJECT

CRESCENDO-FINALS-APPTECH

STACK

React 18, Vite, React Router v6

STORAGE

localStorage frontend persistence

GENERATED

5/12/2026, 3:13:33 AM

1. Fast Explanation You Can Memorize

One-sentence explanation: Crescendo is a React music-learning platform where students log in, choose an instrument, study lessons, take quizzes, save notes, track progress, and unlock harder lesson tiers.

Simple presentation version

Crescendo is a frontend React application built with Vite. It uses React Router v6 for pages, localStorage for accounts and progress, and reusable components for the navbar, progress bars, badges, and metronome. The app has 6 instruments, 3 tiers per instrument, 3 lessons per tier, and 4 quiz questions per lesson. That gives 54 lessons and 216 quiz questions total.

The main learning flow is: log in, choose an instrument, open a lesson, save notes, take the quiz, pass with 70%, and unlock the next tier when all lessons in the current tier are completed.

What to say if asked how the code works

- `main.jsx` starts React and renders `App`.
- `App.jsx` defines all routes and protects pages based on whether a user is logged in.
- `auth.js` saves accounts and current user sessions in localStorage.
- `progress.js` saves lesson completion, notes, review marks, quiz history, progress percentage, tier locks, and ranks.
- `data.js` stores the curriculum: instruments, lessons, videos, diagrams, quizzes, and resources.
- Page files in `src/pages` display the major screens: login, home, dashboard, lesson, quiz, progress, and resources.

2. Project Map

```
C:\Temp\crescendo
├── .github/workflows/deploy.yml    GitHub Pages deployment workflow
├── docs/                          This audit PDF and HTML
├── server/
│   ├── server.js                  Optional future backend API scaffold
│   └── README.md                  Backend notes
├── src/
│   ├── main.jsx                   React entry point
│   ├── App.jsx                    Router and app shell
│   ├── App.css                     Main design system and responsive CSS
│   ├── index.css                   Global root/body setup
│   ├── components/
│   │   ├── Navbar.jsx             Top navigation, user info, logout
│   │   ├── Metronome.jsx           Global rehearsal metronome
│   │   └── Components.jsx          Reusable TierBadge and ProgressBar
│   ├── data/data.js                Instruments, lessons, quizzes, resources
│   ├── pages/                      Login, Home, Dashboard, Lesson, Quiz, Progress, Resources
│   └── utils/                       auth.js and progress.js business logic
├── index.html                      Vite HTML shell
├── package.json                     Scripts and dependencies
└── vite.config.js                   Vite + GitHub Pages base config
```

Ignored/generated areas: `node_modules` and `dist` are generated. The `responsive-check` folder contains previous screenshots and browser profiles; it is evidence, not runtime architecture.

3. High-Level Architecture

Layer	Files	Purpose
Entry	<code>index.html</code> , <code>src/main.jsx</code>	Loads the root div and mounts React.
Shell/Routing	<code>src/App.jsx</code>	Wraps in <code>HashRouter</code> , defines routes, redirects logged-out users, shows global nav/metronome.
Pages	<code>src/pages/*.jsx</code>	Full screens for login, home, lessons, quizzes, progress, resources.
Reusable UI	<code>src/components/*.jsx</code>	Navbar, metronome, badge, progress bar.
Data	<code>src/data/data.js</code>	Static curriculum and resource data.
Utilities	<code>auth.js</code> , <code>progress.js</code>	Accounts, sessions, progress, locks, notes, quiz history, rank.
Styling	<code>App.css</code> , <code>index.css</code>	Black/gold design system and responsive layouts.
Optional Backend	<code>server/server.js</code>	Future API scaffold, not connected to frontend.

4. How the App Starts

1. The browser loads `index.html` .
2. `index.html` loads `/src/main.jsx` in development, or bundled JS after production build.
3. `main.jsx` calls `ReactDOM.createRoot(...).render(<App />)` .
4. `App.jsx` wraps everything in `HashRouter` .
5. `AppShell` checks `getCurrentUser()` from `auth.js` .
6. If a user exists, it renders `Navbar` , protected pages, and `Metronome` . If not, it redirects to `/login` .

index.html → main.jsx → App.jsx → HashRouter → Protected Routes → Page Components

5. Routing and Navigation

The app uses `react-router-dom` v6. It uses `HashRouter` instead of `BrowserRouter` because GitHub Pages can return 404 errors for direct paths like `/login`. Hash URLs like `.../#/login` load the main app first, then React Router handles the route.

Route	Component	Purpose
<code>/login</code>	<code>LoginPage</code>	Sign in, register, demo login.
<code>/</code>	<code>HomePage</code>	Choose an instrument and view standing.
<code>/dashboard/:instrument</code>	<code>DashboardPage</code>	Tier selection and lessons for one instrument.
<code>/lesson/:instrument/:tier/:lessonId</code>	<code>LessonPage</code>	Specific lesson loaded from data.
<code>/quiz/:instrument/:tier/:lessonId</code>	<code>QuizPage</code>	Specific lesson quiz.
<code>/progress</code>	<code>ProgressPage</code>	Progress across all instruments.
<code>/resources</code>	<code>ResourcesPage</code>	External learning resources.

Dynamic route example: `/lesson/guitar/beginner/g-b-1` tells the app to find guitar → beginner → g-b-1 inside `INSTRUMENT_DATA`.

6. Authentication Flow

Authentication is frontend-only. The active React app does not call a server for login. Accounts and sessions are saved in localStorage.

Main files

- `LoginPage.jsx` : form UI, validation, password visibility, register/login mode.
- `auth.js` : storage logic for accounts and current user.
- `Navbar.jsx` : logout button.
- `App.jsx` : route guard.

```
Register:
LoginPage.validate()
→ auth.register({ username, email, password })
→ checks duplicate email
→ creates user with crypto.randomUUID()
→ saves account in crescendo_accounts
→ saves session in crescendo_user
→ navigate('/')

Login:
auth.login({ email, password })
→ searches crescendo_accounts
→ saves session in crescendo_user
→ navigate('/')

Demo:
loginDemo() → saves demo user → navigate('/')

Logout:
Navbar.handleLogout() → auth.logout() → remove crescendo_user → navigate('/login')
```

Security concern: passwords are stored in plain text in localStorage. This is acceptable only for a class frontend demo. A production app needs backend authentication, password hashing, secure sessions, and a database.

7. State Management

The app uses React local state plus localStorage. There is no Redux, Context store, Zustand, Firebase, Supabase, or remote database in the active frontend.

State	Where	Controls
mode , fields , errors	LoginPage.jsx	Form mode, inputs, validation.
activeTier	DashboardPage.jsx	Visible tier lessons.
notes , markedForReview	LessonPage.jsx	Notes and review mark.
answers , result , history	QuizPage.jsx	Quiz selections, result, score history.
bpm , playing	Metronome.jsx	Tempo and start/stop.

```
localStorage keys:
crescendo_user           current logged-in user
crescendo_accounts       registered users
crescendo_progress_demo-student demo account progress
crescendo_progress_<random-user-id> registered account progress
```

8. Data Flow

data.js → pages read lesson/resource objects → progress.js writes user progress → pages re-read progress → UI updates

data.js exports INSTRUMENTS , TIERS , TIER_LABELS , INSTRUMENT_DATA , and RESOURCES . Pages import this directly instead of fetching lessons from a backend.

- 6 instruments.
- 3 tiers per instrument.
- 3 lessons per tier.
- 54 total lessons.
- 216 quiz questions.
- 18 external resources.

Example progress shape:

```
{
  guitar: {
    currentTier: 'beginner',
    beginner: { 'g-b-1': true },
    lessonNotes: { 'g-b-1': 'Practice chord changes' },
    reviewLessons: { 'g-b-1': true },
    quizHistory: { 'g-b-1': [{ score: 100, passed: true, attemptedAt: '...' }] }
  }
}
```

9. User Flow

1. User opens app and lands on login if no session exists.
2. User registers, logs in, or uses demo login.
3. App stores current user and redirects home.
4. User chooses an instrument.
5. Dashboard displays Beginner/Intermediate/Advanced tiers and lesson cards.
6. User opens a lesson, watches video, reads explanation, saves notes, or marks review.
7. User takes quiz.
8. If score is 70% or higher, `markLessonComplete()` saves completion.
9. When all Beginner lessons pass, Intermediate unlocks; when all Intermediate pass, Advanced unlocks.
10. Progress page and navbar rank update based on completion.

10. Feature Flow Explanations

Lesson completion and tier unlock

```
QuizPage.handleSubmit()
→ computes score
→ if score >= 70, markLessonComplete(instrument, tier, lessonId)
→ progress.js marks lesson true
→ checks whether all lessons in that tier are true
→ if yes, updates currentTier to next tier
→ saveProgress() writes localStorage
```

Saved notes

```
LessonPage textarea onChange → updates notes state
Save Notes button → saveLessonNotes()
progress.js writes notes under lessonNotes[lessonId]
Dashboard can show "Notes saved" metadata
```

Quiz history

```
Submit quiz → build review objects → calculate score
→ recordQuizAttempt() → stores latest attempt at top
→ keeps only 8 recent attempts per lesson
```

Rank

```
Navbar → getStudentRank()
→ getStudentProgressSummary()
→ count completed lessons across all instruments
→ show Novice, Apprentice, Performer, Virtuoso, or Maestro
```

11. File-by-File Audit

File	Purpose and connection	Important items	Improvements
src/main.jsx	Mounts React into #root .	ReactDOM.createRoot , StrictMode	Good as-is.
src/App.jsx	Top-level shell, routing, route protection, page transition wrapper, nav/metronome.	HashRouter , Routes , Navigate , getCurrentUser	Could use AuthProvider for centralized login state.
LoginPage.jsx	Login/register screen; calls auth utilities.	validate , handleSubmit , handleDemoLogin	Move validation to utility; avoid frontend password storage.
HomePage.jsx	Instrument selection and current standing.	instrumentImages , getStudentProgressSummary	Move image URLs to data layer.
DashboardPage.jsx	Instrument studio; tier selector and lesson cards.	activeTier , tierSummary , isTierUnlocked	Consider syncing active tier with URL or current unlocked tier.
LessonPage.jsx	Video, explanation, diagram, notes, review, next/previous controls.	getLessonSequence , handleSaveNotes , handleToggleReview	Split into smaller panels.
QuizPage.jsx	Quiz engine, scoring, explanations, history.	PASSING_SCORE , handleSubmit , buildExplanation	Extract scoring to testable utility.
ProgressPage.jsx	Shows completion across instruments and tiers.	getInstrumentCompletion , getTierProgress	Could extract chart/card component.
ResourcesPage.jsx	External resources from RESOURCES .	RESOURCES.map	Add filters later.
Navbar.jsx	Top nav, rank, username, logout.	NavLink , handleLogout , getStudentRank	Context would make rank updates more predictable.
Metronome.jsx	BPM tool using Web Audio API.	useEffect , setInterval , AudioContext	Close/suspend AudioContext on unmount.
Components.jsx	Small shared UI components.	TierBadge , ProgressBar	Rename or split for clarity.
data.js	Static curriculum and resources.	INSTRUMENTS , INSTRUMENT_DATA , RESOURCES	Very large; split by instrument.
auth.js	Frontend account/session storage.	register , login , logout , loginDemo	Replace with backend auth for real app.
progress.js	Learning state and localStorage persistence.	markLessonComplete , isTierUnlocked , recordQuizAttempt	Separate pure reads from writes.
App.css	Main black/gold design and responsive styling.	CSS variables, glass cards, media queries.	Split into smaller CSS files.
server/server.js	Optional Node API scaffold, not connected.	/api/register , /api/login , /api/progress	Add tokens and connect frontend later.
vite.config.js	Vite config and GitHub Pages base path.	base , React plugin.	Change base only if repo name changes.

File	Purpose and connection	Important items	Improvements
deploy.yml	Builds and deploys to GitHub Pages.	npm ci , npm run build , artifact upload.	Add tests/lint later.

12. Backend Communication

Current frontend: no API calls. No `fetch()` , Axios, Firebase, Supabase, or realtime backend is used by React.

Optional backend: `server/server.js` provides a Node HTTP API with JSON file storage, routes for health/register/login/progress, and PBKDF2 password hashing. It is not currently connected to the static GitHub Pages app.

Route	Method	Purpose
/api/health	GET	Health check.
/api/register	POST	Create user.
/api/login	POST	Verify password.
/api/progress	GET/PUT	Load/save progress.

13. Systems Present or Not Present

System	Status	Notes
Firebase/Supabase	Not present	No SDKs/config.
Realtime	Not present	No WebSocket/subscriptions.
Chat/social	Not present	No messaging.
AI	Not present	No AI API calls.
Calendar/tasks	Not present	No planner.
Focus/pomodoro	Not present	Metronome is a practice tool, not a timer.
Gamification	Partial	Ranks, progress bars, tier locks, quiz history, passing celebration.
Pet systems	Not present	No avatar/pet progression.

14. Styling and UI System

The UI is styled mainly in `App.css` . It uses a luxury black/gold design system with CSS variables, glassmorphism cards, gold primary buttons, responsive grids, and page-specific layouts.

- Palette: black, charcoal, metallic gold, muted cream text.
- Typography: Playfair Display headings, Inter body.
- Reusable classes: `glass-card` , `btn` , `btn-primary` , `btn-ghost` , `eyebrow` .
- Responsive rules around 980px, 720px, 460px, and wide desktop.

Concern: `App.css` is 1500+ lines, so it is becoming a high-risk file.

15. Critical Files

Danger level	Files	Why
Dangerous	<code>data.js</code>	Changing lesson IDs can break progress, notes, quiz history, and routes.
Dangerous	<code>progress.js</code>	Controls unlock rules and saved progress.
Dangerous	<code>auth.js</code>	Controls account/session keys and route access.
Dangerous	<code>App.jsx</code>	Controls all routes and redirects.
Careful	<code>vite.config.js</code>	Wrong base path can break GitHub Pages assets.
Safe to refactor	<code>Components.jsx</code> , UI markup	Can split if behavior stays same.
Safe to refactor	<code>ResourcesPage.jsx</code>	Simple static list rendering.

16. Bugs, Dead Code, and Anti-Patterns

Finding	Impact	Recommendation
<code>getLearningEvidenceStats()</code> appears unused after rubric UI removal.	Minor dead code.	Remove if not needed.
Passwords stored in <code>localStorage</code> .	High production security issue.	Use backend auth and hashed passwords.
Progress read functions can write defaults.	Reads have side effects.	Separate read from initialize.
<code>data.js</code> is very large.	Hard to maintain.	Split by instrument.
<code>App.css</code> is very large.	Style coupling risk.	Split CSS by purpose.
Optional backend is not integrated.	May confuse expectations.	Label as future work or connect it.
External images/videos can break.	Third-party dependency.	Use stable local or trusted hosted media.
No automated tests.	Refactors can break logic silently.	Add unit/integration tests.

17. Performance Notes

- Performance is acceptable for the current project size.
- `data.js` bundles all lessons into the main JS; lazy loading by instrument would scale better.
- `useMemo` is used for derived values in several components.
- Repeated progress calculations read `localStorage` often; a central store would be cleaner.
- External images and YouTube iframes are the heaviest resources.

18. Suggested Improved Architecture

```
src/
├── app/
│   ├── App.jsx
│   ├── routes.jsx
│   └── providers/AuthProvider.jsx
├── components/
│   ├── ui/ProgressBar.jsx
│   ├── ui/TierBadge.jsx
│   ├── layout/Navbar.jsx
│   └── practice/Metronome.jsx
├── features/
│   ├── auth/
│   ├── curriculum/
│   ├── quiz/
│   └── progress/
├── data/
│   ├── instruments.js
│   ├── guitar.js
│   ├── piano.js
│   └── resources.js
└── styles/
    ├── tokens.css
    ├── base.css
    ├── components.css
    └── pages.css
```

- Each feature owns its page and logic.
- Data is easier to edit.
- Auth/progress can later switch from localStorage to API calls.
- CSS becomes easier to maintain.

19. Refactor Recommendations

1. Move login validation into `auth.validation.js`.
2. Split `data.js` by instrument.
3. Move quiz scoring into a utility.
4. Add `AuthProvider` for current user state.
5. Add `useProgress()` or a progress provider.
6. Connect backend later for cross-device persistence.
7. Add tests for tier unlock, quiz scoring, and auth validation.

20. Security Recommendations

- Never store real passwords in localStorage.
- Do not trust client-side progress for real grading because localStorage can be edited.
- Use server-side validation and hashed passwords.
- Use secure sessions/tokens and HTTPS.
- Restrict backend CORS. Current scaffold uses `*`.
- Add input validation and request size limits to the backend.

21. Quick Q&A Cheat Sheet

Question	Answer
What framework?	React 18 with Vite.
Where are routes?	<code>src/App.jsx</code> .
Why HashRouter?	To avoid GitHub Pages 404 errors.
Where is state used?	Login forms, dashboard tier, lesson notes, quiz answers/results, metronome BPM.
Where is data stored?	Curriculum in <code>data.js</code> ; accounts/progress in <code>localStorage</code> .
How does unlocking work?	<code>isTierUnlocked()</code> checks every lesson in the previous tier.
How does a lesson complete?	Score at least 70%, then <code>markLessonComplete()</code> .
Does it have backend?	Live frontend does not use one; <code>server/server.js</code> is a future scaffold.
Biggest security issue?	Frontend <code>localStorage</code> auth stores passwords and progress locally.
What improve first?	Real backend auth/progress sync and split large files.

22. Final Mental Model

Think of Crescendo as three connected systems:

1. **Content system:** `data.js` contains instruments, lessons, quizzes, diagrams, and resources.
2. **Learning state system:** `progress.js` records what the student has done.
3. **Navigation/UI system:** `App.jsx` and pages turn content and progress into an interactive app.

Content + Progress + Routes = Crescendo learning experience